

4-Party Asynchronous Communication with Perfect Secrecy based on Parallel Pairs

Introduction

Team Members and Roles:

1. Kanthimathi Sundararajan
 - a. Worked on Informal Description
 - b. Worked on protocol guarantees such as Correctness, Termination and Waste-bound analysis
2. Raymond Lin
 - a. Worked on protocol.py code
 - b. Worked on simulator.py code
 - c. Worked on main.py code
 - d. Worked on Introduction section
3. Thomas Motaïs De Narbonne
 - a. Worked on rigorous description
 - b. Worked on starting proofs (definitions, invariants, proof of invariants)

Protocol Overview

This project attempts to implement and analyze a Parallel Pairs protocol for asynchronous multi-party one-time pad communication with $m=4$ parties. This protocol divides the parties into two independent zones, with each zone containing a pair of parties moving in opposite directions through a shared pad sequence of length n .

Main Claims

Pad Waste:

The following lists the percentage wasted pads among all the usage scenarios in our testing, where n is the number of pads and d is the safety gap, across 100 execution cycles.

Pad Waste Bounds: ($n=1000$, $d=10$, 100 executions)

- S.1 (Single active party, $x=1$): $\sim 52.9\%$ of pads wasted
- S.2 (Two active parties, $x=2$): $\sim 15.5\%$ of pads wasted
- S.4 (Four active parties, $x=4$): $\sim 2\%$ of pads wasted

The maximum number of wasted pads across 100 executions among all the scenarios was 557 out of 1000 pads in the single active party scenario.

Runtime Complexity:

The runtime for a single party to send a message of length L is $O(L)$. This complexity is independent of the number of parties m , total pad sequence length n , and gap length d . The safety check is $O(1)$ constant time, pad consumption is $O(L)$, and collision detection and marking is $O(L)$. Informal Description of the design protocol:

Informal Explanation

Parallel Pairs Protocol ($m=4$)

Think of this as taking four people and splitting them into two completely separate conversations, each happening in its own designated area. Instead of having all four parties compete for the same space, we're essentially running two independent two-person protocols side by side.

Here's the basic idea: we divide our four parties—let's call them Alice, Bob, Charlie, and Dave—into two teams. Alice and Bob form one team, while Charlie and Dave form the other. At the same time, we split our available message pads into two equal halves. Alice and Bob get exclusive access to the first half, while Charlie and Dave work with the second half. The two teams never interfere with each other because they're operating in completely separate zones.

Within each zone, we use the same elegant two-party solution from the lecture notes. Alice starts from the left end of her zone moving rightward, while Bob starts from the right end moving leftward. They send messages only when there's enough safe distance between them—specifically, Alice can send if her next position won't encroach on Bob's last position (minus a safety buffer), and vice versa. Charlie and Dave do exactly the same thing in their own zone.

The beauty of this approach becomes clear when we compare it to the naive solution of just giving each person a fixed quarter of the pads. In that static setup, if only one person talks, you waste 75% of your resources. With parallel pairs, even in the worst case where only Alice talks, she can use her entire half of the pads while only the other half goes unused—that's just 50% waste, which is a significant improvement. When two people from different teams talk (say Alice and Charlie), they each consume their respective zones with almost zero waste.

The tradeoff is simplicity versus flexibility. This protocol is incredibly straightforward to implement—you're literally just running the proven two-party solution twice. There's zero risk of cross-zone collisions. However, it's not load-balanced: if Alice and Bob are having an intense conversation while Charlie and Dave stay silent, Alice and Bob will run out of pads even though half the total capacity sits unused in the other zone. Still, for meeting the core goal of beating the static partition, this protocol delivers with minimal complexity.

Rigorous Description

Party Class

Class_INITIALIZER

- Created a class Party to store each "member" of the party (m=4 so 4 members will be initialized).

Initializes with these argument(s):

- name - Name of party member (ex. Alice, Bob, Charlie, Dave)
- start_index - Where party member starts using pads from
- direction - Whether party member goes left or right (+1 for right, -1 for left), and depends on start_index
- zone_min - Creates minimum range that party member can go (inclusive)
- zone_max - Creates maximum range that party member can go (exclusive)

Initialized with variable(s) but not argument(s):

- current_index - defaults at start_index and increments/decrements based on direction to keep track of where party member is at all times
- has_sent - Keeps track of whether a party member has sent a message or not

get_next_pad_index function

- Returns current_index of specific party member
- Represents next pad index to be used if a message were to be sent

get_last_used_index function

- Goal of this function is to return the last index used by the party member
- We first check whether a message has been sent by the party member, and if not, we simply return their default index which we set as start_index
- Depending on the direction the party member moves, we either return current_index - 1 (if member moving right) or current_index + 1 (if member moving left)
- This ensures boundary of already used pads is always known

can_send function

- Goal of this function is to ensure the party member can properly send a message without violating any rules

Argument(s):

- message_length - Number of pads needed for message
- partner_last_index - Last pad index used by partner in same zone
- d - Safety gap parameter (buffer of unused pads)
- partner_has_sent - Whether party members corresponding partner has sent at least one message

Function

- The direction of the party member helps figure out which branch in the if statement to go (moving right is a positive direction, left is a negative direction)

Right Direction (Positive)

- Store the next start_index (by using current_index) and where the index would end by adding the current_index and the length of the message (subtracted by 1 since its an index)
- Check if next_end is greater than or equal to zone_max, if it is return False since there aren't enough pads for the message
- Check if the party member's partner has sent a message, and check whether there are enough pads between party member and partner. Regardless of whether the partner has sent, enforce $\text{next_end} < \text{partner_last_index} - d$; when partner hasn't sent, partner_last_index equals their start_index.

Left Direction (Negative)

- Store the next start_index (by using current_index minus message_length and plus one cause its an index) and where the index would end by simply setting it to current_index
- Check if next_start is less than zone_min, if it is, return False since there aren't enough pads for the message
- Check if the party member's partner has sent a message, and if he has, check whether there are enough pads between party member and partner. Regardless of whether the partner has sent, enforce $\text{next_start} > \text{partner_last_index} + d$; when partner hasn't sent, partner_last_index equals their start_index.

consume_pads function

- Goal of this function is to consume pad indices for a message once it has been determined safe to send and move the index of talking party member
- Returns a list of pad indices that were consumed

Argument(s)

- message_length - Number of pads needed for message

Function

- The direction of the party member helps figure out which branch in the if statement to go (moving right is a positive direction, left is a negative direction)
- Since the function represents the consumption of pads, the has_sent value is changed to True

Right Direction (Positive)

- Creates a list of the pad indexes that were used to send the message
- Increments the current_index of the party member by the message_length

Left Direction (Negative)

- Creates a list of the pad indexes that were used to send the message
- Decrements the current_index of the party member by the message_length

repr function

- Returns readable string showing:
 - Party member's name
 - Current position
 - Direction (in arrows: --> for right direction and <-- for left direction)

Protocol Class

Class Initializer

- Created a class Protocol to coordinate the full Parallel Pairs system.

Initializes with these argument(s):

- n - Total number of pads in sequence
- d - Gap parameter (max undelivered messages), defaults to 10

Initializes with variable(s) but not argument(s):

- pad_usage - List of length n, filled with booleans to track which pads have been used
- zone_split - n//2 to divide the pad space into two equal zones
- parties - Dictionary to easily access party members

- pairs - Dictionary to easily access party members' partners
- messages_sent - Counts successful sends
- messages_attempted - Counts all send attempts

Partner Making

Zone 1 [0, zone_split):

- Creation of alice using Party class:
 - name - Alice
 - start_index - 0
 - direction - Moving right (+1)
 - zone_min - 0
 - zone_max - zone_split (half of the space)=
- Creation of bob using Party class:
 - name - Bob
 - start_index - zone_split-1
 - direction - Moving left (-1)
 - zone_min - 0
 - zone_max - zone_split (half of the space)

Zone 2 [zone_split, n):

- Creation of charlie using Party class:
 - name - Charlie
 - start_index - zone_split
 - direction - Moving right (+1)
 - zone_min - zone_split
 - zone_max - n
- Creation of dave using Party class:
 - name - Dave
 - start_index - n-1
 - direction - Moving left (-1)
 - zone_min - zone_split
 - zone_max - n

attempt_send function

- Simulates a party attempting to send a message of a given length

Arguments

- party_name - Name of party attempting to send
- message_length - Length of message in pads

Function

- Increment messages_attempted
- Retrieve party member and their partner
- Check if party member is able to send the message properly using can_send function in party class
- If False, return False as message will not be able to be sent without breaking a rule
- call consume_pads with the message_length as the parameter, returning a list of the indexes of used pads
- For each pad index used:
 - Assert pad has not already been used
 - Mark pad as used in pad_usage
 - Increment messages_sent
- Return True, ensuring that no pad is ever reused, preserving perfect secrecy
- Note: The assert not pad_usage[pad_idx] enforces each pad index is used at most once, which is the necessary condition for OTP perfect secrecy.

is_terminated function

- Goal of this function is to check if the protocol has terminated (when no party can send even a minimal message)
- Returns True if protocol is terminated

Function

- For each party member in parties dictionary:
 - Set party and partner for each member and check if they can send a minimal message (length 1) using the can_send function (party class).
 - If any party can send a length-1 message, return False (NOT terminated yet).
- If no party can send a length-1 message, return True (terminated).

get_wasted_pads function

- Return count of unused pads
- Computed as: $n - \text{sum}(\text{pad_usage})$ (since $\text{sum}(\text{pad_usage})$ counts used pads).

get_used_pads function

- Return total count in pad_usage (using sum() function), or the number of used pads

get_waste_percentage function

- Returns $(\text{wasted_pads}/n)*100$ (used get_wasted_pads function for wasted_pads)

get_statistics function

- Returns a dictionary with various statistics about how the protocol went:
 - Total pads
 - Used pads
 - Wasted pads
 - Waste percentage
 - Messages sent
 - Messages attempted
 - Termination status

repr function

- Returns compact summary of:
 - n
 - d
 - Used pads
 - Wasted pads
 - Messages sent

Proofs:

Definitions

Pad Sequence

Let there be a shared ordered sequence of pads:

$$- P = \{p_0, p_1, \dots, p_{(n-1)}\}$$

where each pad may be used at most once.

Parties & Zones

- There are four parties:
 - par1 (Alice)
 - par2 (Bob)
 - par3 (Charlie)
 - par4 (Dave)

The pad space is partitioned into two disjoint zones:

- Zone 1 = $\{0, \dots, (n/2) - 1\}$
- Zone 2 (symmetric) = $\{(n/2), \dots, n - 1\}$

$$\text{Zone}_1 \cap \text{Zone}_2 = \emptyset$$

Initial positions:

- par1 $\rightarrow$ starts at 0, moves right (+)
- par2 $\rightarrow$ starts at $(n/2 - 1)$, moves left (-)
- par3 $\rightarrow$ starts at $(n/2)$, moves right (+)
- par4 $\rightarrow$ starts at $n - 1$, moves left (-)

Each party consumes pads only within its assigned zone.

Used Pads

A pad is considered used at send time. Let:

$$- U \subseteq \{0, \dots, n - 1\}$$

denote the set of pad indices that have been used.

Sending Conditions

Let d be the safety gap parameter.

Let $\text{Last}(X)$ denote the last pad index written by party X .

A party may send a message of length $L \geq 1$ only if:

1. Every pad in the proposed message segment is not in U
2. The entire message segment lies within that party's zone
3. The safety gap condition holds

Safety gap conditions:

In Zone 1:

- par1 may send only if
 - $\text{next_end}(\text{par1}) < \text{Last}(\text{par2}) - d$
- par2 may send only if
 - $\text{next_start}(\text{par2}) > \text{Last}(\text{par1}) + d$

In Zone 2:

- par3 may send only if
 - $\text{next_end}(\text{par3}) < \text{Last}(\text{par4}) - d$
- par4 may send only if
 - $\text{next_start}(\text{par4}) > \text{Last}(\text{par3}) + d$

Invariants

We prove the following invariants hold at all times:

1. Used Pads Invariant
 - a. Every pad index enters U at most once.
2. Zone Separation Invariant
 - a. Each party only writes inside its assigned zone.
3. Safety Gap Invariant (Zone 1)
 - a. $\text{Last}(\text{par1}) + d < \text{Last}(\text{par2})$
4. Safety Gap Invariant (Zone 2)
 - a. $\text{Last}(\text{par3}) + d < \text{Last}(\text{par4})$

Proof of Invariants

Claim

All four invariants hold throughout protocol execution.

We prove this by induction over protocol events.

Protocol events consist of:

- A send event (a party consumes one or more pads)
- A delivery event (does not modify pad indices or U)

Base Case (Time 0)

At time 0:

- $U = \emptyset$

Therefore, the Used Pads Invariant holds.

By construction:

- $\text{par1} \in \text{Zone}_1$
- $\text{par2} \in \text{Zone}_1$
- $\text{par3} \in \text{Zone}_2$
- $\text{par4} \in \text{Zone}_2$

Thus, Zone Separation holds.

We assume the initial configuration satisfies:

- $\text{Last}(\text{par1}) + d < \text{Last}(\text{par2})$
- $\text{Last}(\text{par3}) + d < \text{Last}(\text{par4})$

This holds whenever each zone contains strictly more than d pads,

- i.e., when $(n/2) > d$

Thus, all invariants hold at time 0.

Inductive Step

Assume all invariants hold immediately before some protocol event.

We show they hold immediately after the event.

Case 1: Delivery Event

A delivery event does not modify:

- U
- Any party index

Therefore, all invariants remain unchanged.

Case 2: Send Event

Suppose party X sends a message of length $L \geq 1$.

Let S denote the set of pad indices consumed.

After sending:

- $U := U \cup S$

The party's index advances by L steps in its direction.

Preservation of Used Pads Invariant

Before sending, the protocol verifies:

- Every pad in S is not in U

The implementation enforces this using:

- `assert not pad_usage[pad_idx]`

Thus, no pad index can enter U more than once.

Therefore, the Used Pads Invariant is preserved.

Preservation of Zone Separation

Before sending, the protocol checks:

- Right-moving party: $\text{next_end} < \text{zone_max}$
- Left-moving party: $\text{next_start} \geq \text{zone_min}$

Thus, no party ever writes outside its assigned zone.

Zone Separation is preserved.

Preservation of Safety Gap Invariant (Zone 1)

We analyze Zone 1 and Zone 2.

Assume before sending:

- $\text{Last}(\text{par1}) + d < \text{Last}(\text{par2})$

Case: par1 Sends

Before sending, the protocol checks:

- $\text{next_end}(\text{par1}) < \text{Last}(\text{par2}) - d$

After sending:

- $\text{Last}(\text{par1})_{\text{new}} = \text{next_end}(\text{par1})$

Thus:

- $\text{Last}(\text{par1})_{\text{new}} < \text{Last}(\text{par2}) - d$

Adding d to both sides:

- $\text{Last}(\text{par1})_{\text{new}} + d < \text{Last}(\text{par2})$

The invariant is preserved.

Case: par2 Sends

Before sending, the protocol checks:

- $\text{next_start}(\text{par2}) > \text{Last}(\text{par1}) + d$

After sending:

- $\text{Last}(\text{par2})_{\text{new}} > \text{Last}(\text{par1}) + d$

Thus:

- $\text{Last}(\text{par1}) + d < \text{Last}(\text{par2})_{\text{new}}$

The invariant is preserved.

Preservation of Safety Gap Invariant (Zone 2)

Assume before sending:

- $\text{Last}(\text{par3}) + d < \text{Last}(\text{par4})$

Case: par3 Sends

Before sending, the protocol checks:

- $\text{next_end}(\text{par3}) < \text{Last}(\text{par4}) - d$

After sending:

- $\text{Last}(\text{par3})_{\text{new}} = \text{next_end}(\text{par3})$

Thus:

- $\text{Last}(\text{par3})_{\text{new}} < \text{Last}(\text{par4}) - d$

Adding d to both sides:

- $\text{Last}(\text{par3})_{\text{new}} + d < \text{Last}(\text{par4})$

The invariant is preserved.

Case: par4 Sends

Before sending, the protocol checks:

- $\text{next_start}(\text{par4}) > \text{Last}(\text{par3}) + d$

After sending:

- $\text{Last}(\text{par4})_{\text{new}} > \text{Last}(\text{par3}) + d$

Thus:

- $\text{Last}(\text{par3}) + d < \text{Last}(\text{par4})_{\text{new}}$

The invariant is preserved.

Conclusion

All invariants:

- Used pad uniqueness
- Zone separation

- Safety gap preservation in each zone

hold at time 0 and are preserved by every protocol event.

Therefore, by induction, these invariants hold for the entire execution of the protocol.

Since:

- Every pad enters U at most once
- Each party writes only inside its assigned zone
- Within each zone, the safety gap invariant guarantees that written regions never overlap

It follows that no two parties can ever write to the same pad index.

The protocol therefore guarantees no collision and preserves the one-time pad requirement that each pad is used at most once.

Correctness Proof:

Theorem:

In the Parallel pairs protocol with $m=4$ parties and n pads, no two parties ever write to the same pad index

Proof:

We prove this by partitioning the collision space and showing that collisions are impossible in each partition.

Let:

- A, B, C, D denote the four parties (Alice, Bob, Charlie, Dave)
- $\text{Zone}_1 = [0, n/2-1]$ and $\text{Zone}_2 = [n/2, n-1]$
- $\text{Next}(X)$ denote the next pad index party X will write to
- $\text{Last}(X)$ denote the last pad index party X wrote to
- d denote the safety buffer distance

Case 1: Inter-zone collisions (between parties in different zones)

Parties A and B are restricted to Zone_1 , while parties C and D are restricted to Zone_2

By construction:

- $\max(\text{Next}(A)) \leq n/2-1$
- $\max(\text{Next}(B)) \leq n/2-1$
- $\min(\text{Next}(C)) \geq n/2$
- $\min(\text{Next}(D)) \geq n/2$

Since $\text{Zone}_1 \cap \text{Zone}_2 = \emptyset$, we have:

- $\text{Next}(A) \neq \text{Next}(C), \text{Next}(A) \neq \text{Next}(D)$
- $\text{Next}(B) \neq \text{Next}(C), \text{Next}(B) \neq \text{Next}(D)$

Therefore, no inter-zone collision is possible.

Case 2: Intra-zone collisions (within Zone1)

Alice starts at position 0 moving right ($\rightarrow$), Bob starts at position $n/2-1$ moving left ($\leftarrow$).

The protocol enforces:

- Alice sends only if: $\text{Next}(A) < \text{Last}(B) - d$
- Bob sends only if: $\text{Next}(B) > \text{Last}(A) + d$

Sub-case 2a: Initial state (no messages sent yet)

- $\text{Last}(A) = -1$ (undefined/no previous message)
- $\text{Last}(B) = n/2$ (undefined/no previous message)
- $\text{Next}(A) = 0, \text{Next}(B) = n/2-1$
- $\text{Distance} = n/2-1 - 0 = n/2-1 > 2d$ (given safe initialization)

No collision possible

Sub-case 2b: After k messages have been sent by A and B combined

By induction on the number of messages:

- Base case ($k=0$): Shown in sub-case 2a.
- Inductive step: Assume after k messages, $\text{Last}(A) + d < \text{Last}(B) - d$ (safety invariant holds).

When Alice sends message $k+1$:

- Condition checked: $\text{Next}(A) < \text{Last}(B) - d$
- Alice writes to $\text{Next}(A)$

- New invariant: $\text{Last}(A) < \text{Last}(B) - d$ (by the send condition)
- Therefore: $\text{Last}(A) + d < \text{Last}(B)$, maintaining safe distance

When Bob sends message $k+1$:

- Condition checked: $\text{Next}(B) > \text{Last}(A) + d$
- Bob writes to $\text{Next}(B)$
- New invariant: $\text{Last}(B) > \text{Last}(A) + d$ (by the send condition)
- Therefore: $\text{Last}(A) + d < \text{Last}(B)$, maintaining safe distance

Since both operations preserve the invariant $\text{Last}(A) + d < \text{Last}(B) - d$, we have:

- $\text{Last}(A) < \text{Last}(A) + d < \text{Last}(B) - d < \text{Last}(B)$
- Therefore: $\text{Last}(A) \neq \text{Last}(B)$

By induction, no collision occurs in Zone_1 for any number of messages.

Case 3: Intra-zone collisions (within Zone2)

The proof is symmetric to Case 2:

- Charlie starts at $n/2$ moving right ($\rightarrow$)
- Dave starts at $n-1$ moving left ($\leftarrow$)
- Same safety conditions apply: $\text{Next}(C) < \text{Last}(D) - d$ and $\text{Next}(D) > \text{Last}(C) + d$
- By the same inductive argument, no collision occurs in Zone_2 .

Conclusion:

Since all possible collision cases (inter-zone and both intra-zone cases) are proven impossible, the protocol guarantees that no two parties ever write to the same pad index.

Termination Proof:

Claim:

Every party eventually completes all their messages and terminates.

Proof Structure:

We'll prove termination by showing:

1. Every party can always eventually send their next message
2. Each party has finitely many messages to send
3. Therefore, all parties complete in finite time

Setup:

- n parties total (assume n is even for parallel pairs)
- Parties are paired: $(P_0, P_1), (P_2, P_3), \dots, (P_{n/2-2}, P_{n/2-1})$
- Each party has m messages to send
- Pad space divided into zones:
 - Zone A: indices $[0, N/2)$ for one set of pairs
 - Zone B: indices $[N/2, N)$ for other set of pairs
- Within each zone:
 - One party moves right (increasing indices)
 - Partner party moves left (decreasing indices)

Key Observations:**Observation 1: Finite Message Count**

Each party sends exactly m messages (finite).

Observation 2: Monotonic Progress

- Right-moving parties: pad index strictly increases with each message
- Left-moving parties: pad index strictly decreases with each message
- No party ever revisits a pad index

Observation 3: Safety Condition Always Satisfiable

For a party to send a message of length ℓ :

Right-moving party at position r :

- Needs: $r + \ell \leq \text{partner_position}$
- Partner is left-moving, starting from high index
- Partner's position only moves left over time
- Initially: $r = 0$ and $\text{partner_position} = N/2$ (large gap)

Left-moving party at position l :

- Needs: $l - \ell \geq \text{partner_position}$
- Partner is right-moving, starting from low index
- Partner's position only moves right over time
- Initially: $l = N/2$ and $\text{partner_position} = 0$ (large gap)

Main Argument:

Step 1: Total Pad Consumption is Bounded

Let L = total length of all messages across all parties.

Given sufficient pad space $N \geq L$, we have:

- Total consumption $\leq L \leq N$
- Pad space never exhausted

Step 2: No Deadlock Possible

Suppose party A (right-moving) cannot send message of length ℓ :

- Condition violated: $A.\text{position} + \ell > B.\text{position}$
- This means B must send first to move left and create space
- Can B send? B is left-moving with condition: $B.\text{position} - \ell_B \geq A.\text{position}$

Key insight: The two conditions are complementary:

- If A is blocked: $A.\text{pos} + \ell_A > B.\text{pos}$ $\rightarrow$ gap is small
- B's condition: $B.\text{pos} - \ell_B \geq A.\text{pos}$ $\rightarrow$ B needs smaller gap
- Since initially $B.\text{pos} - A.\text{pos} = N/2$ (large), and total consumption $< N$
- At least one party can always proceed

Step 3: Progress Guarantee

At any point in the protocol:

- If party A is blocked, partner B can send
- If party B is blocked, partner A can send
- If neither is blocked, both can send
- At least one message is always sendable in each pair

This means:

- The system makes progress (messages are sent)
- No infinite waiting
- No circular dependencies

Step 4: Finite Termination

Each party:

1. Sends m messages (finite count)
2. Each message takes finite time to send
3. Never permanently blocked (by Step 3)
4. Makes monotonic progress (by Observation 2)

Therefore:

- Each party completes in at most $m \cdot (\text{max message latency})$ time
- All n parties complete in finite time
- Protocol terminates

Assumptions Required:

1. Sufficient pad space: $N \geq \text{total message length across all parties}$
2. Fair scheduling: No party is starved indefinitely by the scheduler
3. Finite message sizes: Each message has bounded length

Summary

Termination holds because:

- Finite work per party (m messages)
- Monotonic progress (no backtracking)
- No deadlock (complementary safety conditions ensure someone can always proceed)
- Bounded resources sufficient ($N \geq L$)

Waste-bound analysis:

Claim

For a pair (A, B) with total message length $L = \sum \ell_{a,i} + \sum \ell_{b,i}$ operating in a zone of size Z :

- $\text{Waste} = Z - L$

Moreover, the minimum zone size required for termination is:

- $Z_{\min} = L$

Therefore, with optimal allocation ($Z = L$), waste = 0 (the protocol is pad-optimal).

Proof Structure

We'll prove the waste bound by:

1. Defining waste precisely as unused allocated pad indices
2. Tracking gap evolution throughout protocol execution
3. Showing final gap equals waste
4. Proving minimum zone size requirement is exactly L
5. Concluding that optimal allocation achieves zero waste

Setup

- Parties: A (right-moving) and B (left-moving) in their zone
- Initial positions: $r_a(0) = 0$, $l_b(0) = Z$
- Message lengths:
 - A sends messages: $\ell_{a,1}, \ell_{a,2}, \dots, \ell_{a,n}$
 - B sends messages: $\ell_{b,1}, \ell_{b,2}, \dots, \ell_{b,m}$
- Total message length: $L = L_a + L_b = \sum \ell_{a,i} + \sum \ell_{b,i}$
- Zone allocation: Z indices $[0, Z)$ assigned to this pair
- Gap definition: $\text{Gap}(t) = l_b(t) - r_a(t)$ (unused space between parties)

Key Observations

Observation 1: Gap Monotonically Decreases

When A sends message of length ℓ_a :

- A's position: $r_a \rightarrow r_a + \ell_a$
- Gap decreases: $\text{Gap} \rightarrow \text{Gap} - \ell_a$

When B sends message of length ℓ_b :

- B's position: $l_b \rightarrow l_b - \ell_b$
- Gap decreases: $\text{Gap} \rightarrow \text{Gap} - \ell_b$

Every message sent shrinks the gap by exactly the message length.

Observation 2: Total Gap Decrease Equals Total Message Length

Total gap decrease = $\sum \ell_{a,i} + \sum \ell_{b,i} = L$

Since:

- Initial gap: $\text{Gap}(0) = Z - 0 = Z$
- Final gap: $\text{Gap}(\text{final}) = Z - L$

Observation 3: Final Gap Represents Waste

After all messages are sent:

- $\text{Gap}(\text{final}) = Z - L = \text{Waste}$

The remaining gap is the unused pad space that was allocated but never used for encryption.

Observation 4: Gap Must Support Remaining Work

At any point during execution:

- $L_{a,\text{remaining}}$ = total length of A's unsent messages
- $L_{\beta,\text{remaining}}$ = total length of B's unsent messages
- For deadlock freedom: $\text{Gap} \geq \ell_{a,\text{next}}$ OR $\text{Gap} \geq \ell_{\beta,\text{next}}$ (at least one party can send)

From the termination proof, sufficient condition for progress:

- $\text{Gap} \geq L_{a,\text{remaining}} + L_{\beta,\text{remaining}}$

Observation 5: Waste is Order-Independent

The final waste $\text{Gap}(\text{final}) = Z - L$ is the same regardless of the order in which messages are sent.

Message ordering affects:

- When parties can send (timing)
- Which party sends at each step

Message ordering does not affect:

- Total pad usage (always L)
- Final waste (always $Z - L$)

Main Argument

Step 1: Minimum Zone Size Derivation

Question: What is the minimum Z that guarantees termination for any message ordering?

Invariant: Throughout execution:

$$\text{Gap} - (L_{a,\text{remaining}} + L_{\beta,\text{remaining}}) = \text{constant}$$

Proof of invariant:

- Initially: $\text{Gap} - (L_{a,\text{remaining}} + L_{\beta,\text{remaining}}) = Z - L$
- When A sends ℓ_a : Gap decreases by ℓ_a , $L_{a,\text{remaining}}$ decreases by $\ell_a \rightarrow$ difference unchanged
- When B sends ℓ_β : Gap decreases by ℓ_β , $L_{\beta,\text{remaining}}$ decreases by $\ell_\beta \rightarrow$ difference unchanged
- Therefore: $\text{Gap} - (L_{a,\text{remaining}} + L_{\beta,\text{remaining}}) = Z - L$ (constant)

For termination (from deadlock freedom):

- We need: $\text{Gap} \geq L_{a,\text{remaining}} + L_{\beta,\text{remaining}}$ at all times

This requires:

- $Z - L \geq 0$
- $Z \geq L$

Therefore: $Z_{\min} = L$

Step 2: Waste with Minimum Allocation

If we allocate exactly $Z = L$:

- $\text{Waste} = Z - L = L - L = 0$

Zero waste achieved!

Step 3: Waste with Over-Allocation

If we allocate $Z = L + \Delta$ for some safety margin $\Delta > 0$:

- $\text{Waste} = Z - L = (L + \Delta) - L = \Delta$

Waste equals exactly the over-allocation amount.

Step 4: Waste Independence from Execution Order

Claim: Waste is the same regardless of which order messages are sent.

Proof:

The final gap is:

$\text{Gap}(\text{final}) = \text{Gap}(\text{initial}) - (\text{total messages sent})$

$$= Z - (\sum \ell_{a,i} + \sum \ell_{\beta,i})$$

$$= Z - L$$

This calculation is independent of the order in which the individual $\ell_{a,i}$ and $\ell_{\beta,i}$ are summed.

Different orderings may affect:

- Whether certain messages can be sent at certain times (safety conditions)
- The gap at intermediate points
- The timing/latency of completion

But they cannot affect the final total: $\sum \ell_{a,i} + \sum \ell_{\beta,i} = L$ (constant).

Therefore: Final waste = $Z - L$ (independent of order)

Step 5: Optimality

The Parallel Pairs Protocol is pad-optimal:

1. Minimum requirement: $Z \geq L$ (proven in Step 1)
2. Achievable minimum: $Z = L$ works for all message orderings (from termination proof)
3. Zero waste possible: With $Z = L$, waste = 0 (proven in Step 2)
4. No protocol can do better: Any protocol needs at least L pad indices to send total length L

Conclusion: The protocol uses the theoretical minimum pad space.

Assumptions Required

For the waste bounds to hold:

1. Total message length known or bounded: We can allocate $Z \geq L$
2. Fixed zone allocation: Each pair gets a fixed zone size Z (not dynamic reallocation)
3. No cross-zone sharing: Pairs cannot share unused pad space from other pairs
4. Complete execution: All parties send all their messages (protocol terminates)

Summary

Main Results

- Exact waste formula: Waste = $Z - L$ (allocated minus used)
- Minimum zone size: $Z_{\min} = L$ (theoretical minimum)
- Optimal allocation: With $Z = L$, waste = 0 (pad-optimal protocol)
- Over-allocation waste: With $Z = L + \Delta$, waste = Δ (proportional to over-allocation)
- Order-independent: Waste is the same regardless of message sending order

Key Insights

Insight 1: The protocol itself creates no inherent waste—all waste comes from allocation decisions.

Insight 2: With perfect knowledge of total message length L , zero waste is achievable.

Insight 3: Message ordering affects timing/latency but not pad efficiency.

Insight 4: The protocol is as pad-efficient as any possible protocol (theoretical optimum).

Practical Implications

Scenario	Allocation	Waste
Known exact lengths	$Z=L$	0 (optimal)
Bounded by $\ell_{\max}$	$Z = 2m \cdot \ell_{\max}$	Up to $2m(\ell_{\max} - 1)$
Uniform random $[1, \ell_{\max}]$	$Z=2m \cdot \ell_{\max}$	$\approx m \cdot \ell_{\max}$ (50%)
Conservative estimate	$Z=L+\Delta$	Δ

Comparison to Alternative Protocols

Protocol	Minimum Z	Waste (optimal)	Order-dependent
Parallel Pairs	L	0	No
Sequential	L	0	No
Round-robin	$2m \cdot \ell_{\max}$	Variable	No

Conclusion: Parallel Pairs achieves the theoretical optimum for pad efficiency—no waste with optimal allocation, and waste proportional only to over-allocation in practical scenarios.

AI Usage:

We used ChatGPT as a writing aid when writing this report. Specifically, it was used to:

- help refine the wording of informal explanations,
- improve the structure and readability of proof descriptions,
- check consistency in mathematical notation,
- suggest formatting improvements.
- Development of some technical details like class structures and algorithms
- Docstrings for the functions

The protocol design, correctness reasoning, and implementation were developed by the team members.